

JAVA INTERVIEW PREPARATION

CHAPTER 29

GARBAGE COLLECTION

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Garbage Collection: Reachability, Generations, G1, ZGC, and Evidence-Based Tuning

Senior / Lead Java Developer Reading Chapter

Garbage collection automates reclamation, not capacity planning. A collector must find unreachable objects, recover space, compact or manage fragmentation, and coordinate with application threads. Different collectors trade throughput, pause time, footprint, and CPU overhead. Senior engineers begin with service-level goals and GC evidence rather than copying a list of flags from another application.

Reachability, Not Scope

An object becomes eligible for collection when no GC-root path can reach it. Leaving a source-code block does not guarantee immediate reclamation, and System.gc does not promise that a particular object will be collected.

```
roots -> live graph      unreachable graph
      A -> B              X -> Y
      |                   ^ |
      +-> C                +-+
```

X and Y can be collected despite their cycle. A, B, and C remain live as long as the root path remains.

Collectors normally use tracing phases conceptually related to marking reachable objects, reclaiming unreachable space, and sometimes relocating live objects. Exact algorithms differ.

The Generational Hypothesis

Most applications create many objects that die young and fewer that survive for a long time. Generational collectors exploit this pattern by collecting young allocations frequently and long-lived regions less often.

```
allocation -> young objects -> survivors -> older objects
              many die here   some age      fewer remain
```

Generational is a collector design, not a Java language promise. Object age normally means surviving collections, not wall-clock age. Promotion and region rules are implementation-specific.

Allocation and Thread-Local Buffers

HotSpot can allocate many small objects quickly using thread-local allocation buffers, or TLABs. A thread often advances a pointer within its current buffer without a global heap lock.

```
young region
+-- Thread A TLAB: [used...free]
+-- Thread B TLAB: [used...free]
```

Fast allocation does not make allocation free. Objects increase memory bandwidth, GC scanning, copying, and retained-state pressure. Optimize allocation only after profiling; short-lived objects can be cheaper than complex pooling that raises retention and contention.

Stop-the-World and Concurrent Work

During a stop-the-world pause, application threads reach a safepoint and stop while the JVM performs required work. Concurrent collectors move substantial work alongside application execution, but still have some pauses.

```
application: ===|pause|=====|pause|=====
GC concurrent:      ----marking-----
```

"Concurrent" does not mean pause-free, and "low latency" does not mean zero CPU cost. Concurrent work competes for CPU and memory bandwidth and may require headroom to finish before allocation exhausts available space.

Collector Goals

```
throughput: percentage of time doing application work
latency:    pause and response-time distribution
footprint:  heap plus collector metadata and headroom
capacity:   allocation rate and live-set volume supported
```

These goals conflict. A collector optimized for short pauses can use additional barriers, threads, metadata, and headroom. A small batch job may prefer maximum throughput; an interactive API may prioritize tail latency.

Serial and Parallel Collectors

The Serial collector performs collection work with a single GC thread and can suit small heaps or constrained environments. The Parallel collector uses multiple GC threads and emphasizes throughput, usually accepting stop-the-world pauses.

```
Serial:  one GC worker, simple, small-scale use
Parallel: multiple GC workers, throughput-oriented pauses
```

Do not dismiss them as obsolete. Workload, heap size, CPU limits, and latency requirements determine fit. A short-lived batch process may perform well with a throughput collector.

G1: Region-Based Balanced Collection

G1 divides the heap into equal-sized regions rather than requiring one physically contiguous young generation followed by one old generation.

```
G1 heap regions
[E][E][Y][O][O][H][H][F][Y][O]
E empty, Y young, O old, H humongous, F free
```

G1 performs young-only collections and later mixed collections that include selected old regions with substantial reclaimable space. It tracks cross-region references using remembered structures. G1 is the default collector on most Java 21 HotSpot hardware and operating-system configurations.

`-XX:MaxGCPauseMillis` is a pause-time goal, not a hard deadline. G1 selects work to attempt to meet the goal while preserving progress. Setting an unrealistically tiny target can increase overhead and reduce throughput.

Concurrent Marking and Mixed Collections

G1 begins concurrent marking to estimate liveness across old regions. After marking, mixed collections gradually reclaim selected old regions alongside young regions.

```
young collections -> concurrent mark -> remark -> cleanup  
                                     -> mixed collections
```

If marking starts too late or the application allocates faster than G1 can recover space, G1 may experience evacuation failure or fall back to expensive full collection behavior. Heap occupancy, allocation rate, marking duration, and reserve headroom must be considered together.

Humongous Objects

In G1, an object larger than a collector-defined fraction of a region is treated as humongous and occupies one or more contiguous regions.

Large byte arrays, image buffers, serialized payloads, and oversized caches can create fragmentation and special collection pressure.

```
normal objects share regions  
humongous array requires consecutive region capacity
```

Do not tune region size first. Identify allocation sites and ask whether payloads can be streamed, chunked, bounded, or avoided. If the objects are legitimate, collector and region configuration may then be evaluated with evidence.

ZGC and Low-Latency Collection

ZGC performs most marking, relocation, and reference processing concurrently and is designed for very low pause times across large heaps. It uses colored-pointer or metadata techniques and load barriers in HotSpot implementations.

Java 21 introduced Generational ZGC as a separate mode while the existing non-generational ZGC remained available. Exact defaults and option status are JDK-version-specific, so production documentation must name the target JDK.

ZGC needs CPU and memory headroom to perform concurrent work faster than the application allocates. A low-pause collector cannot compensate for a live set that nearly fills the configured memory limit or a container with no native headroom.

Reference Types and Cleanup

Strong references keep objects ordinarily reachable. Soft, weak, and phantom references provide different reachability semantics and are processed by the collector.

Weak references can support canonical mappings where keys should not remain alive solely because of the map. Soft references are not a dependable primary cache-sizing policy; reclamation timing is controlled by the JVM and can cause latency spikes. Phantom references plus a ReferenceQueue can support post-reachability cleanup coordination.

Prefer explicit resource lifecycle with try-with-resources. GC manages memory reachability, not timely closure of files, sockets, transactions, or permits.

Finalization is deprecated for removal and has unpredictable timing and security and performance problems. Use AutoCloseable, Cleaner only when a safety net is genuinely required, and explicit ownership.

Reading GC Logs

Unified logging provides collector evidence.

```
-Xlog:gc*:file=gc.log:time,uptime,level,tags
```

Useful observations include:

- Pause type, duration, and frequency.
- Heap occupancy before and after collection.
- Young and old region changes.
- Concurrent-cycle start and completion.
- Allocation or promotion pressure.
- Humongous allocation activity.
- Full collection, evacuation failure, or allocation stall.
- CPU time used by GC phases.

One long pause is not the same as sustained GC pressure. Correlate logs with request latency, allocation profiles, CPU throttling, container events, and traffic.

Live Set, Allocation Rate, and Headroom

The live set is the memory that remains reachable after effective collection. Allocation rate is how quickly new memory is requested. Both matter.

```
large allocation rate + small stable live set -> frequent young collection  
growing post-GC live set                    -> retention growth or workload growth  
live set near heap maximum                  -> little recovery headroom
```

A larger heap can reduce collection frequency but may increase worst-case work and process footprint. A smaller heap can fail during bursts. Size from measured steady-state live set, traffic bursts, collector needs, and container headroom.

Tuning Workflow

1. Define latency, throughput, footprint, and availability goals. 2. Confirm collector, JDK build, heap limits, container CPU and memory. 3. Capture GC logs and JFR under representative load. 4. Measure allocation rate, post-GC live set, pause percentiles, and CPU. 5. Find application causes such as unbounded retention or humongous allocation. 6. Change one high-value setting or design at a time. 7. Load test and compare against the same baseline.

For G1, official guidance generally recommends beginning with defaults, setting a maximum heap, and adjusting the pause goal only when evidence requires it. Large collections of inherited tuning flags can disable useful ergonomics and encode assumptions from another JDK.

Marking, Barriers, and Remembered References

A tracing collector must find every live object even while application threads continue changing references. Concurrent collectors therefore combine marking with barriers: small pieces of generated code around reference reads or writes that preserve the collector's view of the object graph.

```
application writes: oldObject.field = youngObject
                    |
write barrier records cross-region/cross-generation relationship
                    |
collector scans remembered information instead of the entire old heap
```

G1 uses remembered sets to track references between regions and a snapshot-at-the-beginning style marking protocol for concurrent marking. The implementation details are sophisticated, but the engineering consequence is simple: reference mutation is not free, remembered-set maintenance consumes memory and CPU, and dense cross-region graphs can increase scan work.

The collector also needs brief pauses for operations that require a consistent coordination point, such as initial marking or remarking. Concurrent marking reduces pause work; it does not remove the need to reconcile changes made while marking ran.

Safepoint Time Is Not Always GC Time

Application threads must sometimes stop at a safepoint so the JVM can perform a globally coordinated operation. GC is a major user of safepoints, but class redefinition, deoptimization, biased-lock related historical mechanisms, and other runtime operations can also require coordination depending on the JDK.

```
request to stop
-> time until all required threads reach safe state
-> operation executes
-> application resumes
```

The first interval is time-to-safepoint; the second is operation time. A latency spike attributed to "GC pause" may include delayed arrival at a safepoint, operating-system descheduling, CPU throttling, or another VM operation. Modern unified logs and JFR events help separate these intervals. Always correlate JVM pause evidence with host CPU scheduling and request traces.

G1 Evacuation as the Core Mental Model

During a young or mixed collection, G1 selects a collection set of regions, copies live objects out, updates references, and returns evacuated regions to the free pool.

collection set	survivor/old destinations	reclaimed
[Y: live + dead]	---> copy live objects ----->	[empty region]
[O: mostly dead]	---> copy remaining live ----->	[empty region]

Copying compacts as a side effect, but it needs destination space. If G1 cannot find enough regions to complete evacuation, an evacuation failure can retain regions in place and increase later work. If concurrent marking cannot finish before free capacity is exhausted, the JVM may enter a far more expensive fallback or full collection path.

This makes reserve and timing important. Initiating marking earlier gives more time but consumes concurrent CPU sooner. A larger heap or lower live set gives evacuation headroom. Reducing allocation bursts, humongous objects, or unnecessary retention may outperform any flag change.

Mixed collections do not collect the entire old generation at once. After a successful mark, G1 chooses old regions with favorable reclaimable-space value and includes them across collections while continuing young-generation work. The pause target influences how much work it predicts can fit, but prediction is probabilistic and workload-dependent.

Collector Selection from Workload Goals

small heap / simple constrained process	Serial may be sufficient
batch throughput on multiple CPUs	Parallel is a strong candidate
balanced server latency and throughput	G1 is the usual starting point
very low pause priority and sufficient budget	ZGC is a strong candidate

This is a starting map, not a benchmark result. Evaluate the exact JDK, heap, CPU quota, allocation profile, live set, latency objective, and operational maturity. A collector that delivers shorter JVM pauses can still worsen end-to-end latency if its concurrent threads contend with application work on a CPU-throttled container.

Java 21 supports ZGC and its generational mode. On that release, generational ZGC is explicitly selected with ZGC plus the generational option; later JDK defaults may differ. Deployment documents should record the collector and complete effective command line rather than saying only "we use low-latency GC."

Heap Sizing Is a Feedback System

Heap sizing controls how often the collector must work, how much live data it can accommodate, and how much space remains for bursts and concurrent recovery.

```
required heap ~= steady live set
                + allocation burst allowance
                + evacuation/concurrent-cycle headroom
                + safety margin validated under load
```

Setting `-Xms` equal to `-Xmx` can reduce resizing variability and reserve predictable capacity, but it can also commit a large footprint earlier and reduce density. Leaving them different allows ergonomics to adapt, at the cost of resize behavior. The correct policy depends on container guarantees and workload, not a universal rule.

Do not evaluate heap size from average utilization alone. Measure post-collection occupancy at normal and peak load, allocation-rate percentiles, promotion or old-region growth, and the time required for concurrent cycles. The heap must survive the period between pressure detection and reclaimed-space availability.

Quantifying GC Impact

Useful derived measures include:

```
GC overhead fraction = total GC-related paused/concurrent CPU time / interval
allocation rate      = bytes allocated / second
reclamation rate     = occupancy before - occupancy after effective collection
promotion rate       = bytes surviving into older storage / second
```

Interpret them carefully. Concurrent GC CPU overlaps application time, so it is not equivalent to pause time. A stable live set with extremely high allocation can create frequent young collections without a leak. A low allocation rate with rising post-GC occupancy points more toward retention. Tail pause percentiles and request latency matter more than a single average.

When a GC log says a pause lasted 80 ms while the application reports a 900 ms latency spike, investigate queueing, safepoint delay, CPU throttling, downstream calls, and coordinated traffic effects. GC may be a contributor without explaining the full interval.

Explicit GC and Reference Processing

`System.gc()` is a request, not a guarantee about timing or a particular algorithm. Libraries may invoke it to encourage cleanup of native resources, while diagnostic tools can trigger collection for histograms or dumps. Disabling explicit GC can protect latency but may expose a library assumption; making explicit collection concurrent changes cost rather than removing it. Inspect call sites and the target collector before applying a flag.

Reference processing can also extend pauses or concurrent work when applications create huge numbers of weak, soft, phantom, or finalizable objects. Reference types are lifecycle tools with collector costs, not zero-cost substitutes for ownership. Prefer bounded explicit caches and `AutoCloseable`; use `Cleaner` or phantom-reference infrastructure as a safety mechanism, not the primary release path.

A Strong Interview Explanation

Start with reachability from GC roots and the generational observation that many objects die young. Explain that allocation can be cheap through TLABs while retention, copying, scanning, and promotion determine collection cost. Then compare collectors by throughput, pause distribution, footprint, CPU, and recovery headroom rather than declaring one universally best.

For G1, describe regions, remembered cross-region references, concurrent marking, evacuation, young collections, and mixed collections. State that `MaxGCPauseMillis` is a goal and that evacuation needs free regions. For ZGC, emphasize mostly concurrent relocation and low pauses plus CPU and memory headroom. Finish with the tuning method: establish service goals, collect GC logs and JFR under representative load, measure live set and allocation rate, correct application retention or allocation problems, change one variable, and compare against the same baseline.

Production Perspective

High GC frequency can be the symptom of an allocation storm, undersized heap, memory leak, or workload spike. Long pauses can involve a large live set, reference processing, humongous objects, CPU throttling, or full collection. A low heap percentage does not rule out native-memory pressure.

Do not force `System.gc` as a routine repair. Diagnose why application or library code requests it, and understand whether the selected collector disables, honors, or handles explicit GC concurrently under the target configuration.

Interview-Ready Summary

GC traces reachability from roots and reclaims unreachable memory. Generational designs exploit the tendency of many objects to die young. Allocation can be fast through TLABs, but allocation rate and retained live set still drive collector work.

G1 uses regions, young and mixed collections, concurrent marking, and pause-time goals. ZGC performs most work concurrently for low latency and requires headroom. Collector selection and tuning trade throughput, latency, footprint, and CPU. Start from goals, logs, JFR, allocation profiles, and the post-GC live set rather than folklore flags.

Revision Notes

- Eligibility for collection depends on reachability from GC roots.
- Java GC is not reference counting; unreachable cycles are collectible.
- Generational designs exploit high mortality among young objects.
- TLABs make many allocations cheap but not free.
- Stop-the-world pauses suspend application execution.
- Concurrent collectors still have pauses and consume CPU.
- Throughput, latency, footprint, and headroom trade against one another.
- Serial uses one GC worker; Parallel emphasizes throughput with multiple workers.
- G1 divides the heap into regions.
- G1 performs young-only and mixed collections.
- G1 is the default on most Java 21 HotSpot configurations.
- MaxGCPauseMillis is a goal, not a deadline.
- G1 concurrent marking estimates old-region liveness.
- Humongous objects occupy special region allocations.
- Java 21 offers existing ZGC and a generational ZGC mode.
- Low-pause collection still needs CPU and memory headroom.
- Soft references are not a complete cache policy.
- GC does not replace timely resource closure.
- Finalization is deprecated for removal.
- Read pause duration together with occupancy and application latency.
- Post-GC live-set growth is stronger leak evidence than sawtooth heap usage.
- Start tuning from defaults, evidence, and service-level goals.
- Barriers preserve collector correctness while application references change.
- Remembered sets avoid scanning the whole old heap for young references.
- Safepoint delay and VM operation time are different intervals.
- G1 evacuates live objects and needs free destination regions.
- Mixed collections select reclaimable old regions rather than collecting all old data.
- Collector choice begins with workload goals and must be benchmarked.
- Heap headroom must cover bursts and the collector's recovery interval.
- Concurrent GC CPU is not equivalent to stop-the-world pause time.
- Stable live set plus high allocation is pressure, not necessarily a leak.
- System.gc is a request whose handling depends on collector and configuration.